

# **CliniScan: AI for Chest X-Ray**

**Milestone 4: Full Deployment, Web Architecture, and UI/UX Integration**

<https://cliniscan.rupsikat.tech/>

**Rupsika Thipparthi**

# **Table of Contents**

1. Phase 4 Overview

2. Architectural Paradigm Shift: Decoupled System

3. Frontend Engineering & UI/UX Design

4. Backend API Integration (Flask)

5. Live Deployment & System Testing

6. Conclusion

## 1. Phase 4 Overview

While Milestone 3 focused on rigorous architectural optimization and interpretability (Grad-CAM) of the core PyTorch and YOLOv8m models, Milestone 4 represents the culmination of the 8-week structured pipeline. The primary objective of this final phase was to transition the CliniScan AI from a localized, script-based prototype into a production-ready, fully interactive web application.

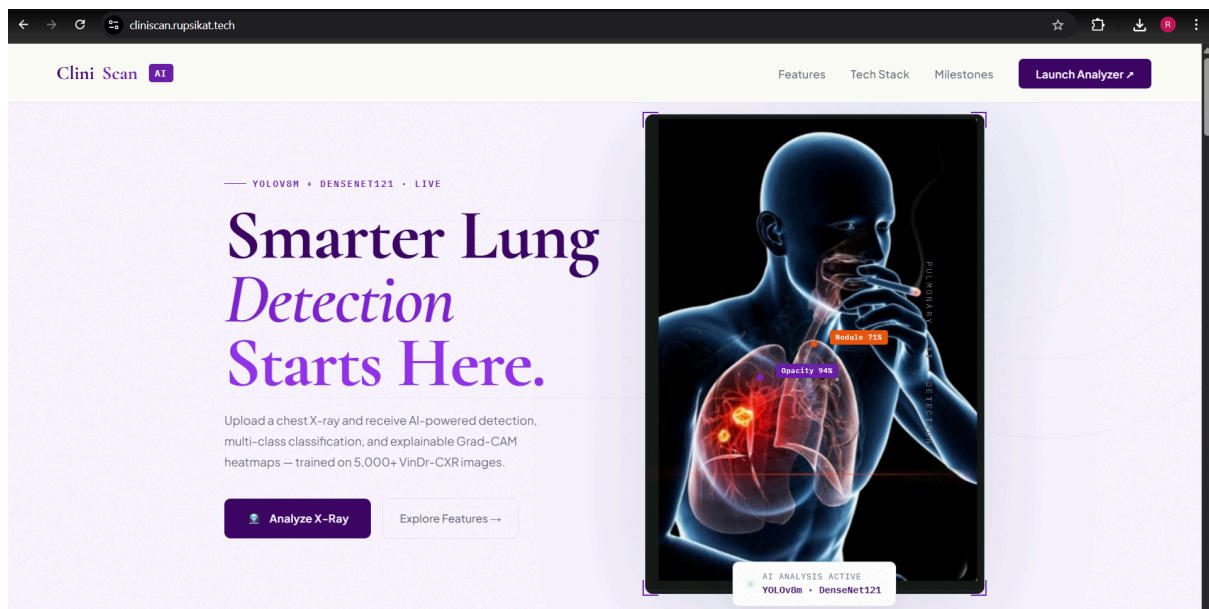
This phase successfully bridged the gap between raw machine learning computation and clinical usability by engineering a custom, decoupled web architecture **deployed on the AWS Cloud**. This robust infrastructure allows users to interact with the 0.4s sub-second inference pipeline securely and in real-time.

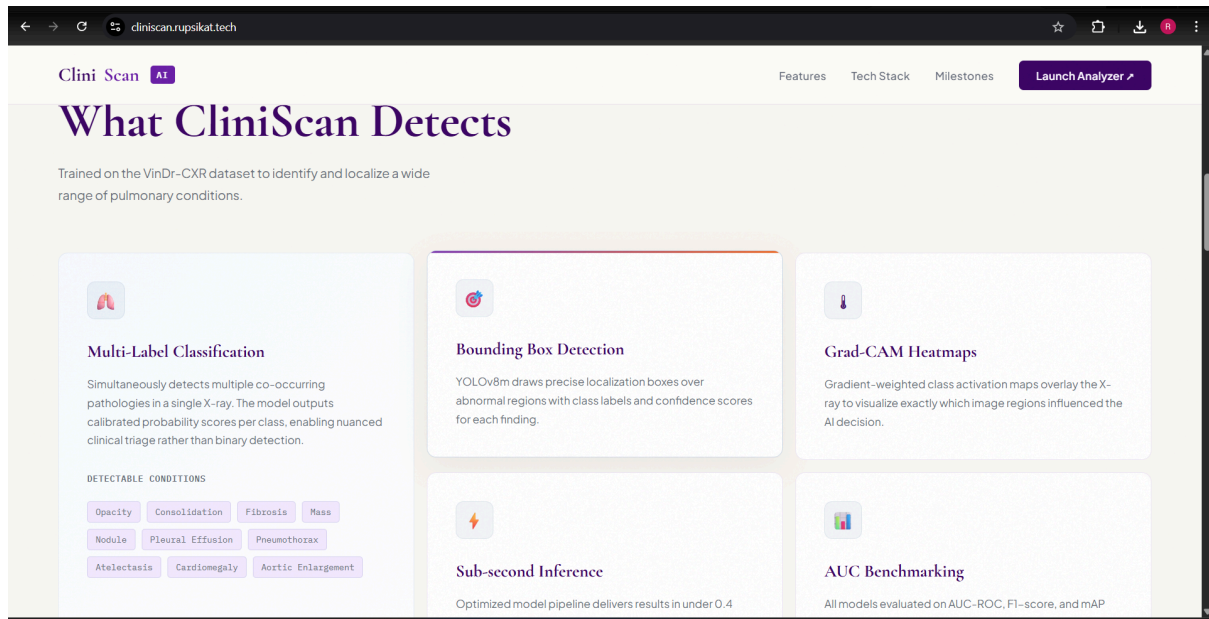
## 2. Architectural Paradigm Shift: Decoupled System

Initially, the project scope considered utilizing Streamlit for rapid deployment. However, to achieve a truly professional, highly customizable, and scalable application, the architecture was upgraded to a strict decoupled paradigm.

- **The Separation of Concerns:** The system was split into an independent client-side interface (Frontend) and an isolated server-side AI engine (Backend).
- **Advantages:** This structural shift eliminated the UI constraints of all-in-one frameworks, allowing for complex DOM manipulations, advanced CSS animations, and asynchronous JavaScript data fetching without interrupting the heavy GPU/CPU tensor calculations happening in the background.

## 3. Frontend Engineering & UI/UX Design





A state-of-the-art, responsive user interface was developed using HTML5, advanced CSS3 (incorporating modern dark-mode aesthetics), and vanilla JavaScript.

Key UI/UX implementations include:

- **Performance Metrics Dashboard:** A dynamic hero section prominently displaying system capabilities, including the 5K+ training images, 14 detectable pathology classes, 0.80 AUC score (DenseNet121), and the 0.4s average inference latency.
- **Interactive Data Visualization:** Custom scroll-reveal animations and interactive hover states (such as the dynamic cursor tracking) were implemented to create a highly engaging user experience.
- **Feature Integration:** The platform clearly delineates the project's core capabilities, visually documenting the Multi-Label Classification, Bounding Box Detection, and Grad-CAM Heatmaps in accessible UI cards.

## 4. Backend API Integration (Flask)

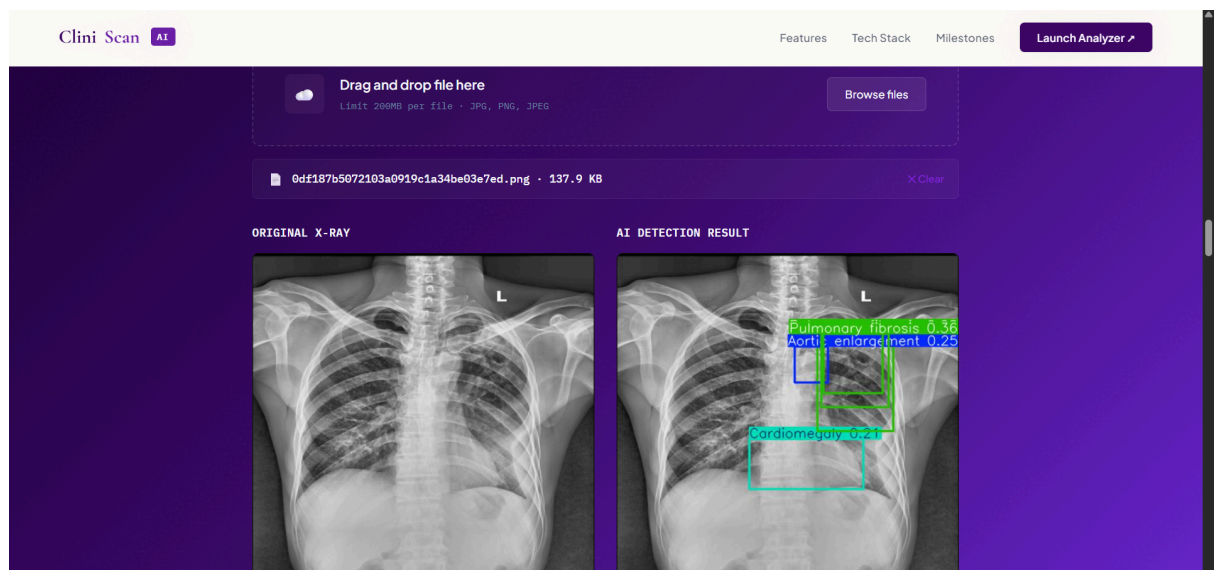
To serve the YOLOv8m detection model to the web, a robust Application Programming Interface (API) was constructed using the Python Flask framework (`app.py`).

- **Silent Inference Engine:** The Flask server operates strictly as a background brain listening on local port 5000. It is completely stripped of UI elements, optimizing it purely for mathematical inference.
- **CORS Protocol:** Cross-Origin Resource Sharing (CORS) was implemented to securely allow the HTML frontend to transmit medical image payloads to the Python backend.

- **Base64 Image Pipeline:** A seamless data bridge was coded using JavaScript `fetch()` APIs. When a user uploads an X-Ray, it is transmitted to the Flask server, processed by the YOLOv8m weights, plotted with OpenCV bounding boxes, and converted into a Base64 text string. This string is returned to the frontend and instantly rendered as an image, preventing the need to save temporary files on the server.

**5. Live Deployment & System Testing** The final deliverable is the Live X-Ray Analyzer module integrated directly into the CliniScan web platform, fully deployed on the **AWS Cloud** to ensure secure, scalable, and high-availability access.

- **Drag-and-Drop Interface:** A custom, hidden file-input architecture was engineered to replace standard browser uploaders, resulting in a sleek "Drag and drop file here" zone.
- **Real-time Diagnostics:** Upon image selection, the system successfully routes the JPG/PNG file to the AWS-hosted backend, triggering the AI model with minimal latency.
- **Dynamic Feedback:** The UI was programmed to handle varying AI responses dynamically. If abnormalities are detected by the YOLOv8m model, the dashboard renders the annotated image with specific confidence scores (e.g., Opacity, Cardiomegaly) localized directly on the thoracic structure.



## 6. Conclusion

Milestone 4 successfully achieved full system deployment. The CliniScan project has evolved from raw VinDr-CXR DICOM files into a highly sophisticated, aesthetically polished, and fully functional diagnostic web application. By leveraging a decoupled Flask and JavaScript architecture, the platform effectively masks the immense complexity of the underlying deep learning models, providing a seamless, sub-second diagnostic experience that aligns with modern software engineering standards.